

In the modern age, web applications are a key part of our everyday lives. Whether it's managing bank accounts, shopping online, communicating, or taking online classes, we're constantly interacting with web programs. However, while these tools make life easier, they come with serious security responsibilities. With every line of code written for the web, there's potential for vulnerabilities that can be exploited by attackers.

As a web developer or anyone interested in building secure systems, it's vital to understand common threats and how to protect users from them. In this essay, we'll explore five of the most significant security risks in web development today. We'll also examine real-world examples where these threats caused damage and discuss practical strategies to mitigate them.

1)SQL injection is a type of attack that occurs when attackers inject malicious SQL code into a query input by way of forms or URL parameters. In the event that the application does not properly handle the input, manipulating the SQL query can do such unintended things as accessing private data, deleting information, and even gaining full control of a database.

It is among the oldest and most common of the forms of attack. Developers habitually use raw SQL queries including unfiltered user input, which represents the consequence.

Heartland Payment Systems: A Real Example:

In 2008, Heartland Payment Systems, a firm processing payments, had a data breach. Access into the system was gained by hackers when they exploited some SQL injection flaws, and they installed spyware that then captured payment card data, as it was being processed. In all, around 130 million credit and debit card numbers were stolen. That is just the total number of cards affected. This kind of breach did tarnish the overall company's reputation, in addition to making them have to pay millions of dollars in various fines and settlements.

SQL commands have the ability to run safely through the use of Prepared Statements since they do separate the query structure away from user input. Injection can be prevented within PHP via functions such as `mysqli_prepare()` or PDO through `bindParam()`. Always check that input matches the expected format, such as numbers for ID fields, for Input Validation as well as Sanitization.

Application-level users should be given zero full access rights inside of the database. Grant only that which is absolutely needed.

Several modern frameworks such as Django or Laravel provide built-in protections against SQL injection.

2)Cross-Site Scripting (XSS)? Cross-Site Scripting is a type of attack in which the attacker injects a script into a web page. When other users load that web page, the script runs in their browser. The script usually does something harmful, like stealing a user's session or displaying a fake login form. Because the script runs with the same privileges as the user that loaded it (i.e., the browser runs it with the same security context), XSS is a very effective way of compromising web applications. There are three types of XSS: Stored XSS: The script is saved on the server and runs when others load the page. Reflected XSS: The script comes from the URL or input and is immediately reflected in the response. DOM-based: The browser executes a malicious script due to insecure JavaScript logic on the page. Real-Life Example: eBay Vulnerability eBay had a problem with stored XSS, which allowed an attacker to embed dangerous JavaScript in product listings. When users would view the listings, the scripts would run in their browsers and redirect them to fake eBay login pages. These unsuspecting users would enter their credentials, thus handing them over to the attackers.

3)Cross-Site Request Forgery (CSRF): What Is CSRF? Cross-Site Request Forgery (CSRF) is when an attacker deceives a person who is logged into a web site into performing some action on another web site without their consent or knowledge. In most cases CSRF is a malicious action such as changing passwords or transferring funds. The attacker can make this happen as the attacked website trusts the browser that it use and not verify that the action or request is actually performed by a legitimate origin. CSRF exploits that the browsers will always send cookies with the request no matter what the origin of the request is. Real World Example: ING Bank Vulnerability In 2008, ING Bank was susceptible to a CSRF vulnerability that could allow hackers to invoke actions to change account settings or transfers funds on the bank's site with CSRF. Hackers could send email, or post URI with embedded CSRF links. If the user was logged into ING Bank and clicked the link, the action would process without the clicker having to confirm it. How to Protect Against CSRF Anti-CSRF Tokens: Generate a unique for each new session and create it to each form. Your server will then check the token before processing. SameSite Cookies: Most moderns browsers support a SameSite cookie attribute. The browser will disallow sending cookies along cross-origin requests. User Confirmation: Very important to always require the user to confirm their request. This also applies to damaging action like transfers or

changing passwords. Control The Request Headers: Compare the request headers. Use the origin to confirm if the request originated on your web site or not.

4)What Is Session Hijacking?

Taking over a session involves stealing the session ID (stored in cookies) and using it to act like the user on a web application. If you have the session token, you can access everything the user can without needing their password.

This frequently occurs over unprotected links (such as HTTP), where cookies are vulnerable to eavesdropping.

Real-Life Example: Firesheep Tool

In 2010, a Firefox add-on named Firesheep was put out for public use, making it possible for anyone in the same Wi-Fi space to unlawfully commandeer sessions on social network sites like Facebook and Twitter. Firesheep did this by capturing session cookies. These are small files that web services use to remember who you are after you've logged in. They contain all kinds of private information, and although they shouldn't, a lot of them were being sent to and from computers over unsecured HTTP connections.

How to Prevent Session Hijacking

HTTPS Must Always Be Used: Encrypted connections stop session tokens from being given away over the network.

HttpOnly and Secure: These two settings ensure that cookies are sent and received only by the intended parties and only when the intended parties are talking over secure channels. JavaScript, for instance, is not one of the intended parties.

Session Regeneration: On login or privilege changes, a new session ID should be generated to prevent fixation attacks. This ensures that an attacker cannot use the session ID to gain access after the session is established.

Logout Automatically: End sessions after a period of no activity.

5)What is a MITM Attack? A Man-in-the-Middle attack occurs when a tertiary party intercepts the communication between a user and a website. The eavesdropper can alter the data and even impersonate the site for sinister purposes. \Example: Lenovo Scandal In 2015, Lenovo sent out laptops loaded with a piece of software named Superfish. Superfish injected advertisements into what should have been secure web traffic. To do this, it

installed a root certificate with which to intercept and decrypt the web traffic. This web traffic was, of course, encrypted with a level of encryption that was (or should have been) secure enough that a piece of ad-injecting malware wouldn't be able to decrypt it. How to:

- Everywhere HTTPS: Use HTTPS with certificates from a trusted Certificate Authority. Enable
- HSTS (HTTP Strict Transport Security): This directs browsers to access your site using HTTPS only.
- Use VPNs on Public Wi-Fi: When on untrusted networks, encrypting all traffic adds a layer of safety.
- Certificate Pinning: This process helps applications recognize when fake certificates are in play.